

STM32 UART Registers and HAL [Not-Examinable]

Ankit Srivastava

EE2028

STM32L4S5xx Datasheet

STM32L4+ User Manual RM0432

B-L4S5I-IOT01A User Manual UM2708



OVERVIEW

- U(S)ART Registers
 - Control Registers
 - Data Registers
 - Status Register
- U(S)ART HAL Driver functions

CONTROL REGISTER 1 (USART_CR1)

Note that FIFO mode is disabled

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	FIFO EN	M1	EOBIE	RTOIE	DEAT[4:0]					DEDT[4:0]				
		rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	CMIE	MME	M0	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	UESM	UE
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Address offset: 0x00, Reset value: 0x0000 0000

- **OVER8: Oversampling mode**
 - 0: Oversampling by 16 (better reliability)
 - 1: Oversampling by 8 (higher speed possible)
- **M1, M0 : Word length**
 - M[1:0] = 00: 8 data bits; 01: 9 data bits; 10: 7 data bits
- **PCE: Parity control enable**
 - This bit selects the hardware parity control (generation and detection). When the parity control is enabled, the computed parity is inserted at the MSB position (9th bit if M=1; 8th bit if M=0) and parity is checked on the received data

CONTROL REGISTER 1 (USART_CR1)

Note that FIFO mode is disabled

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	FIFO EN	M1	EOBIE	RTOIE	DEAT[4:0]					DEDT[4:0]				
		r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OVER8	CMIE	MME	M0	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	UESM	UE
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Address offset: 0x00, Reset value: 0x0000 0000

- PS: Parity selection (see page 1335 of [1] for details)
 - Selects the odd or even parity when the parity generation/detection is enabled (PCE bit set)
- UE = USART Enable; TE = Tx Enable; RE = Rx Enable; xIE = Interrupt Enables

CONTROL REGISTER 2 (USART_CR2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADD[7:0]								RTOEN	ABRMOD[1:0]		ABREN	MSBF RST	DATAINV	TXINV	RXINV
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWAP	LINEN	STOP[1:0]		CLKEN	CPOL	CPHA	LBCL	Res.	LBDIE	LBDL	ADDM7	DIS_NSS	Res.	Res.	SLVEN
rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw			rw

Address offset: 0x04, Reset value: 0x0000 0000

- STOP[1:0]: STOP bits
 - These bits are used for programming the stop bits.
 - 00: 1 stop bit 01: 0.5 stop bit 10: 2 stop bits 11: 1.5 stop bits
- Bits that should be kept cleared when asynchronous mode is used
 - CLKEN: Clock enable; LINEN: LIN (synchronous breaks) mode enable

CONTROL REGISTER 2 (USART_CR2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADD[7:0]								RTOEN	ABRMOD[1:0]		ABREN	MSBF RST	DATAINV	TXINV	RXINV
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SWAP	LINEN	STOP[1:0]	CLKEN	CPOL	CPHA	LBCL	Res.	LBDIE	LBDL	ADDM7	DIS_ NSS	Res.	Res.	SLVEN	
rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw				rw

Address offset: 0x04, Reset value: 0x0000 0000

- Advanced feature bits
 - MSBFIRST: Most significant bit first
 - TXINV: TX pin active level inversion
 - RXINV: RX pin active level inversion
 - DATAINV: Binary data inversion. Only the logical data from/to data register inverted, not start
 - / stop / idle levels
 - SWAP: Swap TX/RX pins

CONTROL REGISTER 3 (USART_CR3)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TXFTCFG[2:0]			RXF TIE	RXFTCFG[2:0]			TCBG TIE	TXFTIE	WUFIE	WUS[1:0]		SCARCNT[2:0]			Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DEP	DEM	DDRE	OVR DIS	ONE BIT	CTSIE	CTSE	RTSE	DMAT	DMAR	SCEN	NACK	HD SEL	IRLP	IREN	EIE
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Address offset: 0x08, Reset value: 0x0000 0000

- ONEBIT: One sample bit method enable
 - Specifies whether a single sample or three samples' majority vote is selected
 - 0: Three sample bit method
 - 1: One sample bit method
 - Selecting the single sample method increases the receiver tolerance to clock deviations
 - When the one sample bit method is selected the noise detection flag (NF) is disabled

CONTROL REGISTER 3 (USART_CR3)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TXFTCFG[2:0]			RXF TIE	RXFTCFG[2:0]			TCBG TIE	TXFTIE	WUFIE	WUS[1:0]		SCARCNT[2:0]			Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DEP	DEM	DDRE	OVR DIS	ONE BIT	CTSIE	CTSE	RTSE	DMAT	DMAR	SCEN	NACK	HD SEL	IRLP	IREN	EIE
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Address offset: 0x08, Reset value: 0x0000 0000

- Bits that should be kept cleared when asynchronous mode is used
 - SCEN: Smartcard mode enable; HDSEL: Half-duplex selection; IREN: IrDA mode enable
- EIE: Error interrupt enable
 - Error Interrupt Enable Bit is required to enable interrupt generation in case of a framing error, overrun error or noise flag (FE=1 or ORE=1 or NF=1 in the USART_ISR register)

HAL > STM32L4XX_HAL_UART.H/C

```
HAL_StatusTypeDef HAL_UART_Init(UART_HandleTypeDef *huart)
{
    if (huart->gState == HAL_UART_STATE_RESET)
    {
        /* Init the low level hardware : GPIO, CLOCK */
        HAL_UART_MspInit(huart);
    }
    __HAL_UART_DISABLE(huart);
    // ((huart)->Instance->CR1 &= ~(0x1UL << (0U)))

    /* Set the UART Communication parameters */
    if (UART_SetConfig(huart) == HAL_ERROR)
    {
        return HAL_ERROR;
    }

    if (huart->AdvancedInit.AdvFeatureInit != UART_ADVFEATURE_NO_INIT)
    {
        UART_AdvFeatureConfig(huart);
    }

    /* In asynchronous mode, the following bits must be kept cleared:
    - LINEN and CLKEN bits in the USART_CR2 register, SCEN, HDSEL and IREN bits in the USART_CR3 register.*/
    CLEAR_BIT(huart->Instance->CR2, (USART_CR2_LINEN | USART_CR2_CLKEN));
    CLEAR_BIT(huart->Instance->CR3, (USART_CR3_SCEN | USART_CR3_HDSEL | USART_CR3_IREN));

    __HAL_UART_ENABLE(huart);      // ((huart)->Instance->CR1 |= (0x1UL << (0U)))

    return (UART_CheckIdleState(huart));
}
```

```
typedef struct __UART_HandleTypeDef
{
    /* UART registers base address
    USART_TypeDef *Instance;
    // UART communication parameters
    UART_InitTypeDef Init;
    // UART Advanced Features init parameters
    UART_AdvFeatureInitTypeDef AdvancedInit;
    // Swap, MSBFirst, Rx/TxPinLevelInvert, etc.

    // transfer parameters
    uint8_t *pTxBuffPtr;
    ...

    #if (USE_HAL_UART_REGISTER_CALLBACKS == 1)
        // callbacks functions
        ...
    #endif
} UART_HandleTypeDef;
```

*Note that the code shown here is taken from the HAL library of another STM32 chip — specifically, the STM32L475. While there might be some minor differences when compared to the HAL library for the STM32L4S5 chip that we use in the lab, the core logic and initialization flow remain the same.

Ideally, the HAL library should be portable across different STM32 chips. However, small variations can exist between versions. These are often introduced due to performance optimizations, software bug fixes, or feedback from developers in subsequent library releases

HAL > STM32L4XX_HAL_UART.H/C

```
HAL_StatusTypeDef UART_SetConfig(UART_HandleTypeDef *huart)
{
    uint32_t tmpreg;

    /*----- USART CR1 Configuration -----*/
    tmpreg = (uint32_t)huart->Init.WordLength | huart->Init.Parity
              | huart->Init.Mode | huart->Init.OverSampling ;
    // UART_MODE_TX_RX =(USART_CR1_TE |USART_CR1_RE) = ((0x1UL << (3U)) |(0x1UL << (2U)))
    // enables both TX and RX
    MODIFY_REG(huart->Instance->CR1, USART_CR1_FIELDS, tmpreg);

    /*----- USART CR2 Configuration -----*/

    MODIFY_REG(huart->Instance->CR2, USART_CR2_STOP, huart->Init.StopBits);

    /*----- USART CR3 Configuration -----*/
    tmpreg = (uint32_t)huart->Init.HwFlowCtl; // hardware flow control setting
    tmpreg |= huart->Init.OneBitSampling;
    MODIFY_REG(huart->Instance->CR3, USART_CR3_FIELDS, tmpreg);

    /*----- BRR (Baud Rate Register) Configuration -----*/
    // BRR details not included in this lecture
    usartdiv = (uint16_t)(UART_DIV_SAMPLING16(pclk, huart->Init.BaudRate));
    huart->Instance->BRR = usartdiv;
}
```

```
typedef struct
{
    uint32_t BaudRate;
    uint32_t WordLength;
    uint32_t StopBits;
    uint32_t Parity;
    uint32_t Mode;
    uint32_t HwFlowCtl;
    uint32_t OverSampling;
    uint32_t OneBitSampling;
} UART_InitTypeDef;
```

*Note that the code shown here is taken from the HAL library of another STM32 chip — specifically, the STM32L475. While there might be some minor differences when compared to the HAL library for the STM32L4S5 chip that we use in the lab, the core logic and initialization flow remain the same.

Ideally, the HAL library should be portable across different STM32 chips. However, small variations can exist between versions. These are often introduced due to performance optimizations, software bug fixes, or feedback from developers in subsequent library releases

RECEIVE DATA REGISTER (USART_RDR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	RDR[8:0]								
							r	r	r	r	r	r	r	r	r

Address offset: 0x24, Reset value: 0x0000 0000

- Bits 31:9 -> Reserved, must be kept at reset value.
- Bits 8:0 -> 9-bit receive data : Receive data value
 - The content from the SIPO (receive) shift register which samples the RX line is copied into this register once a complete byte is received
 - When receiving with the parity enabled, the value read in the MSB bit is the received parity bit

TRANSMIT DATA REGISTER (USART_TDR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	TDR[8:0]								
							rw	rw	rw	rw	rw	rw	rw	rw	rw

Address offset: 0x28, Reset value: 0x0000 0000

- Bits 31:9 -> Reserved, must be kept at reset value
- Bits 8:0 -> 9-bit transmit data : Transmit data value
 - The content is copied into the PISO (transmit) shift register which sends the data serially through TX
 - When transmitting with the parity enabled (PCE bit set to 1 in the USART_CR1 register), the value written in the MSB (bit 7 or bit 8 depending on the data length) has no effect because it is replaced by the parity

INTERRUPT AND STATUS REGISTER (USART_ISR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	TCBGT	Res.	Res.	RE ACK	TE ACK	WUF	RWU	SBKF	CMF	BUSY
						r			r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRF	ABRE	UDR	EOBF	RTOF	CTS	CTSIF	LBDF	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Address offset: 0x1C, Reset value: 0x0200 00C0

Note that FIFO mode is disabled

- Contains various status and interrupt status bits
 - Set by the hardware, generally cleared due to some action by the software
 - Some bits are cleared when appropriate registers are read (e.g., reading USART_RDR clears RXNE) /written (e.g., writing USART_TDR clears TXE as well as TC)
 - Some bits are cleared by writing to Interrupt Clear Register (USART_ICR, not covered in the lecture, but easy). For example, ORE is cleared by writing to ORECF of USART_ICR
 - Some bits are automatically set and cleared (e.g.: BUSY)

INTERRUPT AND STATUS REGISTER (USART_ISR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	TCBGT	Res.	Res.	RE ACK	TE ACK	WUF	RWU	SBKF	CMF	BUSY
						r			r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRF	ABRE	UDR	EOBF	RTOF	CTS	CTSIF	LBDF	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Address offset: 0x1C, Reset value: 0x0200 00C0

Note that FIFO mode is disabled

- If a bit in this register becomes 1, and the corresponding interrupt is enabled in USART_CRx, the interrupt is sent to NVIC and the interrupt handler will be triggered
- There is only one interrupt from each USART interface to NVIC
- If not using interrupts, these bits can be polled by the software as status bits

INTERRUPT AND STATUS REGISTER (USART_ISR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	TCBGT	Res.	Res.	RE ACK	TE ACK	WUF	RWU	SBKF	CMF	BUSY
						r			r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRF	ABRE	UDR	EOBF	RTOF	CTS	CTSIF	LBDF	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Address offset: 0x1C, Reset value: 0x0200 00C0

Note that FIFO mode is disabled

- **BUSY: Busy flag**
 - This bit is set and reset by hardware. It is active when a communication is ongoing on the RX line (successful start bit detected)
 - It is reset at the end of the reception (successful or not)
- **TXE: Transmit data register empty**
 - This bit is set by hardware when the content of the USART_TDR register has been transferred into the transmit shift register
 - It is cleared by a write to the USART_TDR register
 - An interrupt is generated if the TXEIE bit =1 in the USART_CR1 register

INTERRUPT AND STATUS REGISTER (USART_ISR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	TCBGT	Res.	Res.	RE ACK	TE ACK	WUF	RWU	SBKF	CMF	BUSY
						r			r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRF	ABRE	UDR	EOBF	RTOF	CTS	CTSIF	LBDF	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Address offset: 0x1C, Reset value: 0x0200 00C0

Note that FIFO mode is disabled

- **TC: Transmission complete** (both TDR and transmit shift register empty)
 - This bit is set by hardware if the transmission of a frame containing data is complete and if TXE is set
 - It is cleared by software, writing 1 to the TCCF in the USART_ICR register or by a write to
 - the USART_TDR register
 - An interrupt is generated if TCIE=1 in the USART_CR1 register

INTERRUPT AND STATUS REGISTER (USART_ISR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	TCBGT	Res.	Res.	RE ACK	TE ACK	WUF	RWU	SBKF	CMF	BUSY
						r			r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRF	ABRE	UDR	EOBF	RTOF	CTS	CTSIF	LBDF	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Address offset: 0x1C, Reset value: 0x0200 00C0

Note that FIFO mode is disabled

- **RXNE: Read data register not empty**
 - This bit is set by hardware when the content of the receive shift register has been transferred to the USART_RDR register
 - It is cleared by a read to the USART_RDR register. The RXNE flag can also be cleared by
 - writing 1 to the RXFRQ in the USART_RQR (not covered in the lecture) register
 - An interrupt is generated if RXNEIE=1 in the USART_CR1 register

INTERRUPT AND STATUS REGISTER (USART_ISR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	TCBGT	Res.	Res.	RE ACK	TE ACK	WUF	RWU	SBKF	CMF	BUSY
						r			r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRF	ABRE	UDR	EOBF	RTOF	CTS	CTSIF	LBDF	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Address offset: 0x1C, Reset value: 0x0200 00C0

Note that FIFO mode is disabled

- **ORE: Overrun error**
 - Set by hardware when the data currently being received in the shift register is ready to be transferred into the RDR register while RXNE=1. It is cleared by a software, writing 1 to the ORECF, in the USART_ICR register. An interrupt is generated if RXNEIE=1 in the USART_CR1 register or EIE = 1 in the USART_CR3 register
 - Note: When this bit is set, RDR register content is not lost but the shift register is overwritten

INTERRUPT AND STATUS REGISTER (USART_ISR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	TCBGT	Res.	Res.	RE ACK	TE ACK	WUF	RWU	SBKF	CMF	BUSY
						r			r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRF	ABRE	UDR	EOBF	RTOF	CTS	CTSIF	LBDF	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Address offset: 0x1C, Reset value: 0x0200 00C0

Note that FIFO mode is disabled

- **NE: START bit Noise detection flag**
 - Set by hardware when noise is detected on a received frame. It is cleared by software, writing 1 to the NFCF bit in the USART_ICR register
 - This bit does not generate an interrupt during single-buffer communication as it appears at the same time as the RXNE bit which itself generates an interrupt

INTERRUPT AND STATUS REGISTER (USART_ISR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	TCBGT	Res.	Res.	RE ACK	TE ACK	WUF	RWU	SBKF	CMF	BUSY
						r			r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRF	ABRE	UDR	EOBF	RTOF	CTS	CTSIF	LBDF	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Address offset: 0x1C, Reset value: 0x0200 00C0

Note that FIFO mode is disabled

- FE: Framing error
 - Set by the hardware when the stop bit is not recognized on reception at the expected time
 - Caused due to de-synchronization (not sampling bits at the correct time), excessive noise, or a break character (0s for a frame period)
 - It is cleared by software, writing 1 to the FECF bit in the USART_ICR register. An interrupt is generated if EIE = 1 in the USART_CR3 register

INTERRUPT AND STATUS REGISTER (USART_ISR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	TCBGT	Res.	Res.	RE ACK	TE ACK	WUF	RWU	SBKF	CMF	BUSY
						r			r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ABRF	ABRE	UDR	EOBF	RTOF	CTS	CTSIF	LBDF	TXE	TC	RXNE	IDLE	ORE	NE	FE	PE
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Address offset: 0x1C, Reset value: 0x0200 00C0

Note that FIFO mode is disabled

- **PE: Parity error**
 - This bit is set by hardware when a parity error occurs in receiver mode. It is cleared by software, writing 1 to the PECF in the USART_ICR register. An interrupt is generated if PEIE = 1 in the USART_CR1 register

HAL > STM32L4XX_HAL_UART.H/C

```
HAL_StatusTypeDef HAL_UART_Transmit(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t Timeout)
{
    uint32_t tickstart;

    // Init tickstart for timeout management
    tickstart = HAL_GetTick();
    while (Size > 0U)
    {
        // Wait for TDR (transmit data register) to be empty
        if (UART_WaitOnFlagUntilTimeout(huart, UART_FLAG_TXE, RESET, tickstart, Timeout) != HAL_OK)
        {
            return HAL_TIMEOUT;
        }
        huart->Instance->TDR = (uint8_t)(*pData & 0xFFU);
        pData++;
        Size--;
    }

    // Wait for transmit shift register to be empty
    if (UART_WaitOnFlagUntilTimeout(huart, UART_FLAG_TC, RESET, tickstart, Timeout) != HAL_OK)
    {
        return HAL_TIMEOUT;
    }
    return HAL_OK;
}
```

*Note that the code shown here is taken from the HAL library of another STM32 chip — specifically, the STM32L475. While there might be some minor differences when compared to the HAL library for the STM32L4S5 chip that we use in the lab, the core logic and initialization flow remain the same.

Ideally, the HAL library should be portable across different STM32 chips. However, small variations can exist between versions. These are often introduced due to performance optimizations, software bug fixes, or feedback from developers in subsequent library releases

HAL > STM32L4XX_HAL_UART.H/C

```
HAL_StatusTypeDef HAL_UART_Receive(UART_HandleTypeDef *huart, uint8_t *pData, uint16_t Size, uint32_t
                                     Timeout)
{
    uint32_t tickstart;

    // Init tickstart for timeout management
    tickstart = HAL_GetTick();
    while (Size > 0U)
    {
        // Wait for RDR (receive data register) to be non-empty
        if (UART_WaitOnFlagUntilTimeout(huart, UART_FLAG_RXNE, RESET, tickstart, Timeout) != HAL_OK)
        {
            return HAL_TIMEOUT;
        }
        *pData = (uint8_t)(huart->Instance->RDR & 0xFFU); // assuming 8 bit data
        pData++;
        Size--;
    }
    return HAL_OK;
}
```

*Note that the code shown here is taken from the HAL library of another STM32 chip — specifically, the STM32L475. While there might be some minor differences when compared to the HAL library for the STM32L4S5 chip that we use in the lab, the core logic and initialization flow remain the same.

Ideally, the HAL library should be portable across different STM32 chips. However, small variations can exist between versions. These are often introduced due to performance optimizations, software bug fixes, or feedback from developers in subsequent library releases

CONCLUSION

- U(S)ART Registers
 - Control Registers
 - USART_CR1: Control Register 1
 - USART_CR2: Control Register 2
 - USART_CR3: Control Register 3
 - Data Registers
 - USART_RDR: Receive Data Register
 - USART_TDR: Transmit Data Register
 - Status Register
 - USART_ISR: Interrupt and Status Register
- U(S)ART HAL Driver functions